

Week 5: Testing, Debugging and Final Integration

Project: RESQAI — AI-Assisted Humanitarian Disaster Message Classification

System Setup

The final week focused on preparing and testing the complete backend. Dependencies were installed for both the Node.js backend and Python ML service. The trained BERTweet model was to be placed in `ml-service/models/bertweet_model/`. Environment variables were configured for MongoDB, JWT, the ML service URL, the confidence threshold, and the Gemini API key.

Starting the System

The system uses three services: MongoDB, the Python ML service, and the Node.js backend. Health endpoints at `localhost:5000/health` and `localhost:8000/health` can be used to check service availability.

API Testing

The API test workflow includes registering Reporter, Admin, and Rescue Team users; logging in as a reporter; saving the JWT token; creating an incident with a message and location; and retrieving the reporter's incidents using an authenticated request.

Troubleshooting

Potential ML-service issues include a missing model, missing Gemini API key, or Python import errors. Backend issues may include MongoDB being unavailable, a port conflict, or the ML service not running. The health endpoints and direct classification endpoint can be used to isolate problems.

Documentation

Documentation includes `BACKEND_SETUP.md`, `API_EXAMPLES.md`, `backend/README.md`, `ml-service/README.md`, and `PROJECT_SUMMARY.md`. These cover setup, API testing, architecture, and troubleshooting.

Final Outcome

The implementation summary describes a backend with REST APIs, BERTweet classification, Gemini fallback, MongoDB persistence, JWT authentication, role-based access control, incident lifecycle management, rescue-team assignment, validation, and security measures. The summary reports 31 files, 3,000+ lines of code, and 15+ API endpoints. The trained model must be present and environment variables correctly configured for the system to run as intended.